

Problems

PROBLEMS:

1. Internet connectivity, the passengers don't have access to a good network so computing can't be done near their end, so we need to build a server in between to serve this purpose.
2. Illiteracy
3. Women and children safety, driver behaviour monitor or tracker, or safety of routes. An alert system which notifies the nearby buses or passengers, and feedback system
4. Most of the buses, despite being assigned for the rural areas, do not take that path and take other illegal routes. An alert system which will let us monitor this 24/7.
5. Occupancy of the buses
6. Navigating through rural areas is not very easy especially with various connecting buses in between and the roads.
7. Alternative options for the passengers due to bus breakdowns
8. Connectivity issue from bus to server (lets say GPS information isn't available at that moment) so being able to predict its arrival time.
9. Information from the past so that it allows us to predict future position of the buses more accurately as well as the average speed of the bus and its arrival time.
10. Passenger demand

FEATURES TO BE INCLUDED:

Nithila

1. Bus Tracking Hardware

Each bus will be equipped with a low-cost tracking unit consisting of:

- ESP32 microcontroller
- GPS module for real-time location tracking
- GSM/4G communication module
- MicroSD card for local data storage during network outages
- Backup battery for uninterrupted operation

The device continuously collects the bus's location, speed, and timestamp. When internet connectivity is available, the data is transmitted to the cloud server. In low-connectivity zones, the data is temporarily stored on the SD card and automatically synchronized once connectivity is restored.

2. Passenger Information through PWA

For passengers with smartphones, a Progressive Web App (PWA) will provide real-time bus information without requiring installation from an app store.

User Inputs:

- Starting Point
- Destination

Information Displayed:

- Available buses serving the selected route
- Estimated Time of Arrival (ETA)
- Current bus location
- Next stop of the bus
- Delay status (On Time / Delayed)

The PWA is lightweight, works on low-end smartphones, and can function efficiently even under limited network conditions.

3. Passenger Information through SMS

Recognizing that many rural passengers still use feature phones, an SMS-based service will provide the same essential information without requiring internet access.

SMS Query Format:

Bus-specific query

BUS 45A

Response

Bus 45A

Current Location: Samayapuram

Next Stop: No.1 Tollgate

ETA: 12 minutes

Route-based query

ROUTE KATTUR TRICHY

Response

Available Buses:

45A - ETA 12 min

12B - ETA 25 min

34C - ETA 41 min

This ensures accessibility for passengers regardless of smartphone ownership.

4. ETA Prediction Engine

The backend server processes incoming GPS data and calculates:

- Current bus position
- Next stop
- Estimated arrival time

Initially, ETA can be calculated using distance and average route speed. As more historical data is collected, the system can incorporate:

- Time-of-day traffic patterns
- Historical travel times
- Market-day and school-hour congestion to improve prediction accuracy.

5. Transport Authority Dashboard

A web-based dashboard will provide transport authorities with operational insights such as:

- Live fleet monitoring
- Route adherence tracking
- Delay analysis
- Fleet utilization statistics
- Driver performance monitoring
- Tamper and connectivity alerts

This enables data-driven decision-making and improves overall fleet efficiency.

6. Offline-First Connectivity Strategy

When connectivity is unavailable:

1. GPS data is stored locally on the SD card.
2. Tracking continues uninterrupted.
3. Once connectivity is restored, buffered data is automatically uploaded to the server.

This ensures continuous route monitoring even in remote locations

Feasibility Analysis

Bus-side tracking unit

Cloud-based backend server

Passenger and authority interfaces (PWA, SMS, and dashboard)

1. Bus-Side Hardware Feasibility

Proposed Hardware

MSPM0G3507 Microcontroller

GPS Module

SIM7600 4G Communication Module

MicroSD Card

Tamper Detection Circuit

Battery Monitoring Circuit

Responsibilities of the Tracking Unit

The MSPM0G3507 acts as an edge device responsible for:

- Reading GPS coordinates
- Parsing location information
- Creating structured tracking records
- Monitoring network connectivity
- Storing records locally during network outages
- Synchronizing stored records when connectivity returns
- Detecting GPS failures
- Detecting tampering events
- Sending periodic health and heartbeat packets

Cost Feasibility

Estimated cost per bus:

Component	Estimated Cost (₹)
MSPM0G3507	150–300
GPS Module	300–700
SIM7600 Module	1200–2500
MicroSD Storage	100–200
Power Circuitry	100–300

PCB & Enclosure	300–700
Installation	300–500
Total	~₹4,000 per bus

For approximately 22,800 government buses:

Estimated deployment cost: ₹9–11 Crore

2. Power Feasibility

The MSPM0G3507 is a low-power microcontroller and consumes only a small fraction of the total system power.

Major power consumers are:

SIM7600 communication module

GPS module

Therefore, the microcontroller does not significantly impact the overall power budget and is suitable for continuous operation inside buses.

3. Server Feasibility

Fleet Size

Tamil Nadu operates approximately:

22,800 Government Buses

Across:

38 Districts

Expected Tracking Load

Assuming:

One location update every 30 seconds

Updates per second:

$22,800 \div 30 \approx 760$ updates/second

This is a relatively small workload for modern cloud infrastructure.

Recommended Architecture

Instead of deploying one server per district, a centralized cloud architecture is recommended.

Benefits:

- Lower infrastructure cost
- Easier maintenance
- Centralized data management
- Better scalability
- Simplified software updates

Architecture:

All Buses → Cellular Network → Central Cloud Server → Passenger Services

4. Passenger Accessibility Feasibility

To maximize accessibility, multiple access channels are proposed:

Progressive Web App (PWA) for smartphone users.

Features:

- Source and destination search
- Live bus tracking
- ETA prediction
- Next stop information

SMS Service for feature phone users.

Example:

BUS 45A

Returns:

1. Current location
2. Next stop
3. ETA

WhatsApp Chatbot (Future Enhancement)

Since WhatsApp is widely used, a chatbot can provide a familiar interface for passengers.

Benefits:

- No new app installation
- Familiar user experience
- Easy adoption across urban and rural populations

5. Estimated Operating Costs

Annual Costs

Item	Estimated Annual Cost
Cellular Data Plans	₹1–2 Crore
Cloud Infrastructure	₹0.1–0.4 Crore
SMS Services.	₹0.3–0.8 Crore
Maintenance	₹0.5–1 Crore

Estimated Total: ₹2–4 Crore per year

Component-Wise Power Consumption

Component	Estimated Power Consumption
MSPM0G3507	~0.03 W
NEO-6M GPS Module	~0.12–0.17 W
A7670 LTE Module (Average)	~0.6–1.2 W
microSD Card	~0.05 W (average)
LM2596 Converter Losses	~0.15 W

Average System Power

The estimated average power consumption of the complete tracking unit is: $\approx 1\text{--}1.5\text{ W}$

Under normal operating conditions, the LTE module contributes the largest share of power consumption, followed by the GPS module. The microcontroller itself consumes only a small fraction of the total power budget.

Peak Power Consumption

During LTE network registration and data transmission bursts, the A7670 module may draw significantly higher current.

Estimated peak system power: $\approx 8\text{--}10\text{ W}$

These peaks occur only for short durations and do not represent continuous operation.

Bus Battery Impact

Assuming a 12 V vehicle power supply:

Average current draw ≈ 100 mA

Peak current draw ≈ 0.8 –1 A

This load is negligible compared to the electrical capacity available in a commercial bus and therefore does not significantly impact vehicle operation.

Energy Consumption

At an average power consumption of approximately 1.2 W:

Daily energy consumption ≈ 29 Wh/day

Monthly energy consumption ≈ 0.86 kWh/month

This corresponds to a very small operating energy.

Harshini

SMART RURAL BUS TRACKING SYSTEM – PROTOTYPE FLOW

Stage	Component	Role	Output
1	GPS Module	Get bus location, speed and time	Latitude, Longitude, Speed
2	IR Sensor Pair	Count passenger entry and exit	Passenger Count
3	TI MSPM0 MCU	Read GPS and IR data, calculate occupancy, create data packet, run self-diagnosis	Bus Data Packet
4	Memory Buffer	Store data when network is unavailable	Buffered Data
5	LoRa / GSM / WiFi	Transmit bus data to server	Live Updates
6	Server / Cloud	Store data, calculate ETA, generate analytics	ETA, Occupancy, Bus Status
7	Dashboard	Fleet monitoring and analytics	Live Bus Tracking
8	Display Board	Show passenger information	ETA, Occupancy
9	IVR System	Provide information through phone call	Voice-based ETA Information

BUS DATA PACKET

Bus ID
Current Location
Current Speed
Passenger Count
Occupancy %
Timestamp
Health Status

Buck Converter

Bus Battery (12V/24V)



Buck Converter



Regulated Output Voltage



MSPM0 + GPS + A7670

TI MCU Selection

Selected:

- MSPM0G3507

Reason:

- Multiple UARTs for simultaneous GPS and 4G communication
- ADC for battery monitoring
- Flash memory for temporary data storage(**128 KB flash**)
- Timers for periodic updates
- Watchdog for reliability
- Low power and low cost

Data Packet Design

Defined packet structure:

```
{  
  "bus_id":"BUS_01",  
  "route_id":"ROUTE_05",  
  "latitude":17.9785,  
  "longitude":79.5941,  
  "speed":35,  
  "timestamp":"...",  
  "battery":"82%",  
  "Occupancy": "43"  
}
```

Offline Data Storage Logic

Designed:

Network Available



Send Data

Network Lost



Store Data

Network Returns

↓
Resend Stored Data

Bus Battery Measure

Bus Battery

↓
Voltage Divider

↓
ADC Pin of MSPM0

↓
ADC Hardware (inside MCU)

↓
Digital Value

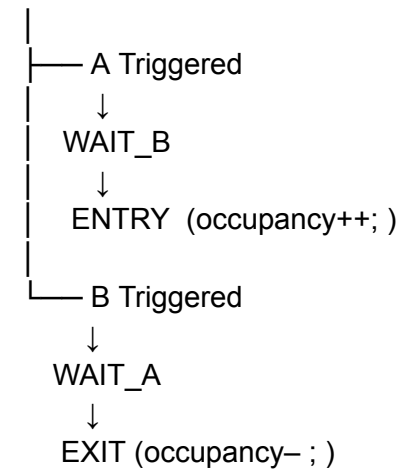
↓
Your C Program

Occupancy detector - dual IR Sensors

Distance between sensors:

5 cm to 15 cm

IDLE



Output :

```
{
  "bus_id": "BUS_01",
  "occupancy": 43,
  "overcrowded": true
}
```

Cost and size :

Component	Approx Cost (India)	Size (Approx)	Important Parameters
NEO-6M GPS	₹300 to ₹600	25 × 25 mm	Accuracy, satellites, update rate, UART
MSPM0G3507 LaunchPad	₹2000–₹2500	65 × 50 mm	Flash, UARTs, ADC, GPIOs
A7670 LTE Module	₹1000–₹1500	30 × 30 mm (module)	LTE support, current consumption, SIM support
LTE Antenna	₹100–₹300	Varies	Signal strength
LM2596 Buck Converter	₹50–₹150	43 × 21 mm	Input range, output voltage, current capability
SIM Card	Existing SIM	-	Data plan
Total	4k to 5k	palm size	

Priyanka

MAIN THEME

Rural passengers face difficulty because they do not know the real time location, arrival time, route and delay status of buses. Poor network connectivity, lack of proper tracking, safety concerns, low affordability, illiteracy makes the problem even more serious.

PROBLEMS

Apart from the problems already discussed some other problems are:-

- 1)The driver may turn off or tamper the device so there should be a way to detect sudden tampering. This can also ensure better safety.
- 2)Route updation. Sometimes roads may be blocked due to landslides, weather, construction etc.
- 3)Another point mentioned in the PS was that the device should be ultra low cost. So for that instead of expensive trackers maybe we can use basic components like GPS module, GSM module Bus battery or power supply a microcontroller. This will reduce the device cost. Also instead of sending GPS data every second we can send the data after some interval, maybe 2-3 minutes. This reduces internet cost.

Out of all the problems the **PRIORITY**

- 1) Providing real bus location and arrival time information in low-network rural areas at low cost.

So core problem to solve is Bus tracking + low connectivity fallback + prediction

Priority	Problem	Why it matters
1	No accurate bus arrival time	This is the biggest daily pain point for passengers. So firstly a proper bus route and a proper time should be decided for the bus to reach a particular stop and that schedule should be displayed on a screen or spoken at the bus stop.
2	Poor network connectivity	Rural areas may not have continuous internet/GSM, so normal tracking fails. In that case sms should be used
3	Low-cost requirement	If the system is expensive, local bus operators will not adopt it.
4	Smartphone/internet access problem for passengers and Illiteracy	One should add display boards, audio or sms
5	Prediction when live data is missing	This makes the system reliable even when GPS/GSM updates fail. And gives an estimate
6	Route deviation detection	Important for safety and monitoring, but after basic tracking works.

7	Safety alerts for women/children/passengers	Can install buzzers alarms in the bus
8	Driver behaviour monitoring	Useful, but not first priority.
9	Passenger demand prediction	
10	Overcrowding/seat availability	

Block wise solution

Bus Unit

- 1) GPS Module (NEO-6M)
- 2) ESP32
- 3) GSM module(SIM800L/SIM7600)
- 4) SIM card
- 5) Optional SD card to store data when network is unavailable
- 6) Buck converter
- 7) Push buttons/buzzers for emergency

Server Unit

The server stores bus data, detects network failure, predicts location, and calculates ETA

Problems solved

- Bus cannot send live data continuously
- Need to predict location using previous location
- Need to calculate arrival time
- Need to store route and stop data
- Need to show live or estimated status

Bus Stop unit

Problems solved

- Passengers without smartphones
- Poor mobile internet at rural stops
- People do not know when the bus will arrive
- Elderly passengers may not use apps
- Low-cost public information access

Microcontroller, GSM module, LCD display, battery. There can also be a mobile app for users with phones apart from the bus stop display

Building up on the 8th point(Fallback prediction)

For that the bus routes should be fixed at first and should be known by the people managing And the bus sends location something like the bus number location time speed and maybe using all the data points calculate the average speed for better prediction

To detect network failure, the server can check the current time-last updated time. If no update comes more than a threshold time start the prediction mode

Last location = Stop A

Last update time = 10:05 AM

Current time = 10:10 AM

Time passed = 5 minutes

Algo

1. Store the fixed route and average travel time between stops.
2. Keep receiving live GPS data from the bus.
3. If live data is received, update the bus location normally.
4. If live data stops, check how much time has passed since the last update.
5. Use the last known bus position and move it forward along the route according to average speed/time.
6. Show this location as "Estimated Location".
7. If no live update comes for too long, stop prediction and show "Last known location".
8. When the network returns, compare actual location with predicted location and correct the bus position.

This will also give us metrics in checking as in the template it was mentioned. Like we can have a metric of prediction accuracy.

BUS UNIT

This is to collect the bus location, process it and send it to the server and if the network is not available it should store the location temporarily.

Part	Component	Purpose
Controller	ESP32 / ESP8266 / Arduino or any Ti microcontroller + GSM	Controls the whole bus unit
Location	GPS module, like NEO-6M	Gets latitude, longitude, speed, time
Communication	GSM module, like SIM800L / A7670, or LoRa if needed	Sends data to server
Power	Bus battery + buck converter, like LM2596	Converts bus 12V/24V to 5V/3.3V
Storage	Internal flash / SD card optional	Stores location when network fails
Status indicators	LEDs / buzzer optional	Shows GPS/network/error status

Data	Which device gives it?	How?
Bus ID	Microcontroller	Fixed in code, for example BUS_01
Latitude	GPS module	GPS gives current location
Longitude	GPS module	GPS gives current location
Speed	GPS module	GPS directly gives speed, or microcontroller calculates it
Timestamp	GPS module / Server	GPS gives time, or server adds time when data is received
Route ID	Microcontroller	Fixed in code, for example ROUTE_05
Battery/status(optional)	Microcontroller	Reads battery voltage using voltage divider and ADC pin

- **For data storage**

1) ESP32 internal flash memory: ESP32 has built-in flash memory. We can store small amounts of data inside it. For our project, this is enough for a prototype. We can store each location packet like this:

```
{  
  "bus_id": "BUS_01",  
  "route_id": "ROUTE_05",  
  "latitude": 31.7754,  
  "longitude": 76.9861,  
  "speed": 35.2,  
  "timestamp": "2026-06-05 16:10:00",  
  "battery": "82%"  
}
```

2) SD Card module: to store a lot of data we can use SD card. The data can be stored in csv format.

Algorithm

Start

Read GPS data

Create packet:

Bus ID + Route ID + Latitude + Longitude + Speed + Timestamp + Battery

Check network

If network available:

Send packet to server

If sending successful:

Continue normal operation

If sending failed:

Store packet locally

If network not available:

Store packet locally

After some time:

Check network again

If network comes back:

Send stored packets one by one

Delete packet only after server confirms received

Repeat

Microcontroller to use (TI microcontrollers that can be used)

1) TI MSPM0G3507 LaunchPad: it can handle GPS reading, battery sensing, temporary storage control, GSM/4G module control, data packet creation.

TI's MSPM0 SDK supports common peripherals like **GPIO, I2C, SPI, UART, ADC, timers**, etc., which are exactly what we need for GPS, GSM, battery reading, and storage.

2) TI CC3220SF LaunchPad: It has built-in Wi-Fi, so for a prototype we can send data using Wi-Fi directly. TI describes the CC3220SF LaunchPad as a Wi-Fi SimpleLink wireless MCU development kit with **1 MB flash and 256 KB RAM**. But for a real rural bus system, Wi-Fi may not always be available, so we may still need a GSM/4G module.

1. Read GPS data using UART
2. Extract latitude, longitude, speed, and timestamp
3. Add bus ID and route ID
4. Read battery voltage using ADC
5. Create data packet
6. Check GSM/4G network
7. If network is available, send data to server
8. If network is not available, store data locally
9. When network returns, upload stored data

Module to microcontroller communication:

Connection	Protocol	Meaning
GPS module → TI MCU	UART	GPS sends latitude, longitude, speed, time
GSM/4G module ↔ TI MCU	UART	MCU sends AT commands to send data
SD card ↔ TI MCU	SPI	Store unsent data temporarily
Battery sensor → TI MCU	ADC	Read battery voltage

Bus to server communication

The TI microcontroller sends commands to the 4G module using **UART**. The 4G module connects to the mobile network and sends data to the server.

Use 4G/LTE module like A7670, SIM7600, SIM070

Power Supply

Device	Required voltage
TI microcontroller	3.3V
GPS module	3.3V or 5V depending on module
GSM/4G module	Usually around 3.8V to 4.2V
SD card module	3.3V or 5V depending on module

Final components

1) NEO-6M GPS module:-

POWER SUPPLY:-The NEO-6M chip needs between 2.7 and 3.6 volts to operate. Fortunately, the module includes a MICREL MIC5205 Ultra-Low Dropout 3.3V regulator. Even better, the logic pins on the module are 5V-tolerant, which means you can connect it directly to an Arduino or any other 5V microcontroller without needing a level shifter.

POWER SAVING:-The module uses about 45mA of current during normal operation. However, it also has a feature called Power Save Mode (PSM), which lets it turn off parts of the chip when they're not needed. In PSM, it can use as little as 11mA, making it perfect for battery-powered devices like GPS watches or trackers.

RECHARGABLE:- The module includes a 4KB HK24C32 EEPROM chip and a small rechargeable button battery. The battery charges by itself when the module is powered on and can keep the saved data for about two weeks even without power. Has antenna on board

Some functions of the module:-

- `gps.speed.value()`: This function gives you the current ground speed, measured in hundredths of a knot. For example, if it returns 1234, that means the speed is 12.34 knots. **SO THE MICROCONTROLLER WOULD HAVE TO CONVERT SPEED INTO km/hr**
- `gps.course.value()`: This function returns the current ground course, or the direction in which the device is moving. It's also given in hundredths of a degree. So a value of 12345 would mean 123.45 degrees.
- `gps.satellites.value()`: This tells you how many satellites the GPS module can see and is using for location tracking. More satellites usually means more accurate positioning.
- `gps.hdop.value()`: This gives you the Horizontal Dilution of Precision (HDOP). It's a number that shows how accurate your GPS location is. Lower numbers mean better accuracy.
- `gps.age()`: This function tells you how old the data is—how long it has been since the GPS module last updated the information. It gives the age in milliseconds. If the value is 1500 or more, it could mean the GPS has lost its signal (called a "lost fix").

- 2) MSPM0G3507 MCU:- Has 4 UART interfaces so we can connect GPS and GSM at the same time without conflict. The GPS module sends raw location data to the MSP. The MSP reads that data and extracts latitude, longitude, speed and timestamp. Then MSP sends the final packet to the GSM/4G module using another UART. It has a timer if we want periodic location updates. It has up to 128KB flash memory so we can store unsent data in that in a prototype and for a real bus system external flash or SD card could be used. Has other GPIOs for LEDs, buzzers, control signals etc. It has an ADC to monitor battery voltage status. Has a watchdog timer to reset the system if it hangs.
- 3) A7670 4G LTE module:-The A7670 series supports LTE Cat-1 and also supports LTE-TDD, LTE-FDD, GSM, GPRS, and EDGE. It supports up to 10 Mbps downlink and 5 Mbps uplink, which is more than enough for sending small bus location packets. It should not be powered directly from the MSP board. Power supply range is 3.4V to 4.2V. Bus battery 12V/24V -> Buck converter -> Stable 4V supply for GSM/4G module
- 4) Buck Converter/ Regulator
- 5) For a prototype we can use Power bank, LiPo battery, 9V/12V rechargeable battery. But for a real system prefer bus battery(seperate battery needs charging, maintenance increases, capacity issues)
- 6) LTE Antenna
- 7) Protection components
- 8) SD card/external flash
- 9) LTE antenna:- connected to 4G/LTE module so that it can catch mobile network signals. We need it so that module connects to module network without antenna.
- 10) Enclosure box

The microcontroller collects data from the GPS module, adds bus identity and battery status, creates a data packet, and sends it to the A7670 4G module. The A7670 module transmits this packet to the server through the mobile network. The server stores the data, calculates ETA, and makes it available to the passenger app or bus stop display.

Shreya

Problems

1. Aging Bus Fleet and Financial Constraints

Many rural transport corporations continue operating old buses because upgrading or replacing vehicles is expensive. On routes with low passenger demand, revenue often does not cover fuel, maintenance, and operational costs.

Example:

A 50-seat bus may carry only 10–15 passengers on certain rural routes, making it difficult for operators to justify investments in modernization.

Impact:

- Higher breakdown rates
- Unreliable service
- Increased maintenance costs
- Poor passenger experience

2. Seasonal and Environmental Disruptions

Rural transportation is heavily affected by environmental conditions.

Common Causes:

- Monsoon flooding
- Muddy roads
- Road construction
- Cattle crossings
- Landslides in some regions

Example:

A route that normally takes 15 minutes may require 45 minutes during adverse weather conditions.

Impact:

- Unpredictable arrival times
- Missed connections
- Reduced service reliability

3. Lack of Formal Bus Stops

Many rural passengers do not wait at designated bus stops.

Instead, they often gather at:

- Village entrances
- Temples
- Tea stalls
- Local landmarks

Drivers may routinely stop at these locations even though they are not officially recorded.

Impact:

- Difficulty tracking passenger demand
- Inaccurate route information
- Challenges in providing precise arrival notifications

4. Student Transportation Challenges

School and college students rely heavily on public buses for daily travel.

Issues:

- Missing classes due to delays
- Exam attendance problems
- Difficulty planning travel schedules

Potential Extension:

Schools could receive automated alerts when buses are delayed.

5. Last-Mile Information Gap

Even if passengers know the location of a bus, they may still face difficulties.

Examples:

- They may be 1–2 km away from the pickup location.
- They may not know where the bus will stop.
- Many villages lack clearly marked bus stops.

Impact:

Tracking alone does not guarantee passengers can successfully board the bus.

6. Uneven Fleet Utilization

Some buses remain in operation for most of the day while others are significantly underutilized.

Example:

- Bus A operates 14 hours/day.
- Bus B operates only 3 hours/day.

Impact:

- Inefficient resource allocation
- Increased operational costs
- Reduced overall fleet productivity

7. Service Verification Problem

GPS tracking can confirm that a bus passed a location, but it cannot confirm whether service was actually provided.

Example:

GPS Record:

Bus reached the stop.

Reality:

Bus passed without stopping.

Impact:

Transport authorities may incorrectly assume service was delivered.

Proposed Solution

1. On-Bus Smart Tracking Unit

Each bus is equipped with:

- GPS Module for real-time location tracking
- GSM Module for communication
- Local Storage for offline operation
- Microcontroller (TI MCU) for data processing

The system collects:

- Location coordinates
- Speed
- Route adherence information
- Delay information

If network connectivity is unavailable, the device stores data locally and uploads it once connectivity is restored.

2. Rural Communication Layer

Since reliable 4G/5G coverage is not always available in rural regions, the system primarily relies on GSM networks.

Information can be delivered through:

- SMS-based queries
- Missed-call service
- IVR (voice calls in local languages)
- Display boards at major bus stops

To reduce costs and maintenance:

- Solar-powered displays can be used.
- Displays can be installed only at major transit points rather than every stop.

3. Offline Route Navigation System

Instead of depending on online navigation services, route information is stored locally inside the bus unit.

The onboard system contains:

- Route coordinates
- Stop locations
- Alternate routes

Using GPS data, the system verifies that the driver remains on the assigned route and can detect route deviations.

4. Intelligent ETA Prediction

Rather than estimating arrival times solely from distance, the system uses historical route information and real-time vehicle data.

Inputs include:

- Current speed
- Historical delays
- Seasonal road conditions
- Route characteristics

This enables more accurate prediction of bus arrival times, especially in rural environments where travel times vary significantly.

Vedika

Empathize —

Three user groups whose pain points drive every decision:

- **Passengers** — rural commuters, often women, elderly, students. Wait 1–3 hours with no information. Many cannot read English route boards and do not own smartphones.
- **Drivers and conductors** — operate without supervision; may deviate from the assigned route; have no easy channel to report breakdowns or blockages.
- **Depot managers** — run fleets blind. No visibility into route adherence, delays, or driver behaviour. Cannot afford commercial trackers (₹6,000–₹10,000 per bus).

The common thread is *absence of trustworthy information*, made worse by patchy cellular coverage, costly hardware, and a non-literate, non-smartphone user base

Define — The problem we are solving

Live tracking, passenger information at stops, and route analytics for operators — most existing solutions only address the passenger side. Our project serves three stakeholders (passenger, driver/conductor, depot manager) through a single shared data pipeline. This is what state transport corporations would actually adopt at scale.

In scope: live position, arrival info at the stop for illiterate users, route adherence for the depot, ETA improvement over time.

Core Technical Decision — Connectivity

Three candidate approaches:

- **GSM / 2G with SMS fallback** — cheapest and most widely deployable; SIM800L module; works on any rural SIM.
- **(MAYBE) LoRa mesh** — low-power link between buses and stops, ~10 km line-of-sight range; buses act as data mules carrying updates from dead-zone stops to internet-connected stops; zero recurring cost.
- **Hybrid** — GSM primary, LoRa fallback, on-bus storage when both fail; directly addresses the pointer on predicting position when GPS/network drops.

Feature Set Mapped to the Pointers

- **Connectivity gaps** → on-bus edge buffer with store-and-forward sync (Basically storing data onboard when in poor range areas, and once you do get in range transfer this buffered data as well.)
- **Illiteracy** → voice and icon-based stop display in regional language; colour-coded route numbers.
- (Maybe, since this will require actual data of viable routes) **Route adherence** → geofence around assigned route; alert depot on deviation beyond threshold.
- **Navigation** → connection suggestion engine (e.g. "Bus 17 in 12 min connects to Bus 41 at Junction X"). (but gotta think more about the literacy point since planning intricate routes as in M-indicator requires some comfort with smartphone usage)
- **Breakdowns** → auto-rebroadcast nearest alternative when a bus goes silent beyond a threshold.

- **Historical learning** → per-segment speed profile by hour and day, used as ETA base.

Differentiators

- **Buses as data mules** — LoRa store-and-forward for dead-zone stops.
- **Probabilistic ETAs** — return a confidence range rather than a single time; honest under uncertainty.
- **Conductor as a sensor** — a single "leaving now" button on the driver console; more reliable than GPS dwell-time inference and friendly to low-literacy operators.

4. Prototype Deliverables

- Cardboard mock-up of a rural bus stop with a phone playing the passenger UI loop.
- Figma flows for passenger app, driver tablet, and depot dashboard.
- (MAYBE)Python + Folium simulation replaying a synthetic GPS trace on a real rural route in Maharashtra, comparing ETA accuracy with and without the historical-learning layer.
- The simulation is the centrepiece – full system demonstrable without hardware.

5. Differentiators

- **Buses as data mules** — LoRa store-and-forward for dead-zone stops.
- **Probabilistic ETAs** — return a confidence range rather than a single time; honest under uncertainty.
- **Conductor as a sensor** — a single "leaving now" button on the driver console; more reliable than GPS dwell-time inference and friendly to low-literacy operators.

GSM Pipeline –

Stage 0: On the Bus (Data Capture)

Hardware on board: GPS module (NEO-6M) + microcontroller (ESP32) + GSM module (SIM800L) + small SD card + a "leaving now" button on the driver console.

What gets captured, and how:

- **Latitude, longitude, speed, heading** — read from the GPS module once per second over a serial line. The GPS chip itself computes these by trilaterating from satellite signals. (Note: look into this)
- **Timestamp** — taken from the same GPS reading (GPS time is accurate even without internet).
- **Bus ID and route ID** — hard-coded into the ESP32 firmware at install time.
- **Driver button press** — a GPIO interrupt when the conductor presses "leaving stop now."

What is done locally before sending:

- **Sampling reduction:** even though GPS reads every second, we only *transmit* every 15–30 seconds, or immediately when the bus crosses a stop geofence, or instantly on a panic press (keeping panic press function on hold).
- **Compression:** instead of sending JSON, we pack the payload as a fixed binary frame (~20 bytes: bus ID, lat, lon, speed, heading, timestamp, flags). This matters because 2G data is slow and metered.
- **Buffering:** every frame is appended to a local SD-card queue *before* any transmission attempt. This is the store-and-forward guarantee.

Stage 1: How the bus actually sends its location to the server

The setup, in everyday terms:

- The ESP32 is the *brain* of the device. It collected the location from the GPS chip and packed it into a tiny 20-byte message.
- The SIM800L is the *phone* of the device. It has a SIM card in it and can connect to a cell tower, just like your phone.
- The brain and the phone are wired together with a small cable (UART = just a two-wire serial connection — think of it as a private wire between two chips).

What happens step by step:

1. **Brain → Phone:** Every 15–30 seconds, the ESP32 sends data to the SIM800L.
2. **Phone connects to the internet:** The SIM800L turns on its mobile data (this is what "GPRS / 2G data session" means — same idea as you turning on mobile data on your phone). Now it's online and has an internet connection.
3. **Phone sends the message to the server:** It sends the 20-byte message over the internet to our server. We use a tool called **MQTT** to send it. Think of this MQTT thing as a *messaging app designed specifically for tiny devices on bad networks*— keeps polling if signal is flickering.
4. **The "topic" idea:** When the bus sends the message, it doesn't say "send this to *that specific server program*." Instead, it shouts the message under a label like:
bus/route5/B17/telemetry
4. This label is called a **topic**. On the server side, any program that says "*I want to listen to all bus telemetry*" will automatically receive it. It's like a group chat — the bus posts to the group, anyone subscribed to the group gets it. The bus doesn't need to know who's listening.

When the network fails:

- If there's no signal, the SIM800L can't send. So the ESP32 just **saves the message to the SD card** instead. It keeps adding new messages to a queue on the card.
- Every 30 seconds it tries again. As soon as signal comes back, it sends every saved message in the right order. Each message still has its *original* timestamp inside it — so when the server eventually receives them, it knows "*this is where the bus was at 10:32, and this is where it was at 10:33*", even though both arrived at 10:45.
- **SMS fallback for emergencies:** If the data connection has been dead for over 5 minutes and something urgent happens (panic button), the SIM800L sends a plain text message — like a normal SMS — to a phone number
we own. SMS works in many places where mobile data doesn't, because it needs less signal.

Stage 2: What the server does when a message arrives

The setup: the "server" is just a small rented computer running on the internet (a VPS — basically a low-cost cloud machine, costs ~₹500/month). It runs a few small programs:

- One program listens for MQTT messages (the **MQTT broker**, called *Mosquitto*).

- One program listens for incoming SMS.
- One program (the **ingestion worker**) takes whatever they receive and processes it.

What happens for each message that comes in:

1. **"Is this bus one I know about?"** — The server has a list of registered buses. If the message claims to be from a bus that isn't on the list, it gets thrown away. (Prevents random fake messages.)
2. **Unpack the message** — Remember the message arrived as 20 bytes of compressed binary? The server reverses that and pulls out the individual values: bus ID, latitude, longitude, speed, etc.
3. **Sanity check** — Does this make sense? If the speed says 800 km/h, or the latitude says the bus is in the Pacific Ocean, something is wrong with the GPS — discard the message. This catches bad readings before they confuse anyone.
4. **Save it** — Write the cleaned-up info into a database. We use a **time-series database** — this just means a database that's specially designed for *"timestamped events arriving constantly"*, which is exactly what we have. Each row is one snapshot of one bus at one moment.
5. **Tell everyone who cares** — The server immediately broadcasts the new info to other parts of the system that need to react right away (the passenger app, the depot dashboard, etc.).

Stage 3: Turning raw GPS into useful information

A raw GPS coordinate isn't useful on its own. Three small programs run continuously to turn it into something passengers and depots actually want.

- *The map-matching **automated program*** runs on the server and snaps GPS points to the route.
- *The ETA **automated program*** runs on the server and recalculates arrival times.
- *The anomaly **automated program*** runs on the server and watches for off-route or silent buses.

Stage 4: Historical Learning Loop (Pointer 9)

Once a day:

- A batch job reads the previous day's telemetry from the time-series DB.
- For each route segment between adjacent stops, it computes the median travel time bucketed by hour-of-day and day-of-week.
- These values overwrite the "expected speed" table used by the ETA worker the next morning.
- Over a few weeks, the system learns realistic speeds for that specific route — much better than relying on Google's generic estimates for rural roads.

Quick Mental Model

[GPS + sensors on bus]

↓ once per second

[ESP32 buffer + SD card]

↓ every 15–30 s, or on event

[SIM800L → 2G/GPRS → MQTT] (SMS if GPRS fails for >5 min + SOS)

↓

[Ingestion server: auth, decode, validate, store]

↓

[Real-time workers: map-match, ETA, anomaly]

↓

[Passenger app | Smart stop display | Depot dashboard]

↓ nightly

[Historical learning job → updates ETA model]

Reethika

More Problems:

1. Long Waiting Times , can be solved If users have access to schedules Of buses
2. Seasonal Route Disruptions Few routes may become impassable during seasons like monsoons.
3. Accessibility , no accommodation for visually/hearing-impaired users in current systems.

POSSIBLE SOLUTION:

1. On the Bus: Gathering the Data

Inside the rural bus, the On-bus hardware layer acts as the data gatherer.

- GPS Module (NEO-6M): Constantly tracks the exact latitude, longitude, and speed of the bus out on the rural roads.
- Occupancy Sensor (INA219): Keeps a live count of how many passengers are on board so people waiting down the line know if there's space left.
- Panic Button: A dedicated safety switch for emergencies that uses a "GPIO interrupt" (meaning it instantly tells the brain to drop everything and handle an alert).
- CC3220SF MCU (The Brain): This TI microcontroller collects all this info (location, passenger count, safety status) and packages it together.

2. Over the Air: Sending it from the Countryside

Because rural areas often have terrible internet, the system doesn't rely on standard heavy mobile data.

- It uses GSM (basic cellular networks) or LoRa (Long Range radio) to beam this packaged data over long distances across the countryside to the main server.

3. At the Edge Server: Processing & Predicting

The Edge Server is the central powerhouse receiving the data. It does the heavy thinking:

- Kalman Filter ETA Prediction: Instead of just guessing, it uses smart math to calculate exactly when the bus will arrive at the next stop, factoring in speed and delays.
- Geofence Engine: It monitors if the bus is sticking to its rural route or if it has accidentally deviated into the wrong area.

4. To the Commuters & Operators: The Outputs

At the Rural Bus Stop (Stop Display): An affordable, low-power LCD/OLED screen at the physical bus shelter shows commuters the Next Bus, ETA, and switches languages so locals can read it.

On Personal Phones (IVR / SMS): For commuters without smartphones, they can call a basic number or get a text message. The system speaks the ETA directly to them in their local language (like Telugu). This is a massive feature for accessibility in rural India!

At the Central Office (Depot Dashboard): Transport operators see a live web map with a moving dot representing the bus. If a driver presses the panic button, a flashing alert pops up here immediately

SLIDE 4:

1. Empathize: Understanding your users

Key Stakeholders to Interview

Passengers

- Rural commuters
- Women passengers
- Elderly passengers
- Students
- Farmers
- Daily wage workers

Questions to be asked?

How long do you usually wait for buses?

Have you ever missed a connection because of delays?

Do you own a smartphone?

What information do you wish you had while waiting?

What safety concerns do you face?

Insights from Empathy

- Passengers need certainty more than precision.
- Many users do not have smartphones.
- Women feel unsafe during long waits.
- Authorities lack route adherence data.
- Network coverage is unreliable.

2. DEFINE

The lack of predictable bus information is a healthcare access crisis, not just an inconvenience.

It would be game-changing if people could hear (not read) a reliable estimate of when the next bus arrives, in her local language.

Need

People need reliable bus arrival information because waiting for long periods wastes time and money.

Insight

Passengers care less about exact tracking and more about knowing whether a bus is:

- Coming soon
- Delayed
- Cancelled

Problem Statement

Rural passengers need a reliable way to know bus arrival times and route status even in areas with poor internet connectivity because uncertainty causes missed appointments, economic loss, and safety concerns.

3.Ideate:

Idea 1: The "Smart Bus Stop"

A solar-powered electronic display installed at rural bus shelters that receives ETA data via GSM/LoRa and shows it to passengers.

- **Pros:** * Completely solves problems like passengers don't need smartphones or internet.
 - Highly visible, providing a safe gathering point for women and children
- **Cons:** * High upfront cost to install hardware at thousands of rural shelters.
 - Risk of theft, vandalism, or damage due to harsh rural weather conditions.
- **Key Inference: Takeaway:**

A single Dash board under the authority for monitoring.

Idea 2: Passenger Mobile App with Offline/SMS Tracking

An Android app that works offline by allowing passengers to send a free missed call or automated SMS to a server to get an instant reply with the bus location.

- **Pros:** * Zero infrastructure cost at the bus stops.
 - Works on basic feature phones (JioPhones) without needing active 4G data .
- **Cons:** * Excludes passengers who are completely illiterate or don't own a phone at all (often women or elderly in deep rural pockets).

- **Key Inference:** * *Takeaway:* While an app/SMS system shouldn't be the *primary* solution. We need something which can be easily accessible by all the people irrespective of their education.
IVR can be used for this.

Idea 3: The Edge-Prediction (Server-Side Logic)

A smart backend server that uses historical data to predict ETAs when GPS drops and automatically flags drivers taking illegal routes

Pros: * Directly solves route deviation and tracking issues without requiring extra hardware on the bus. And also works if the network is lost.

Uses a data queue to handle massive loads .

Cons: * Doesn't solve the passenger-facing problem on its own; the data is stuck in the cloud/depot.

- **Key Inference / Final Implementation Point:** * *Takeaway:* This software layer is mandatory. It acts as the "brain" of the entire system. It will process the raw GPS coordinates, handle dead zones using store-and-forward logic, and then *feed* the refined ETA data directly to the physical Display nodes and also send out sms and ivr from Idea 1,2.

Server

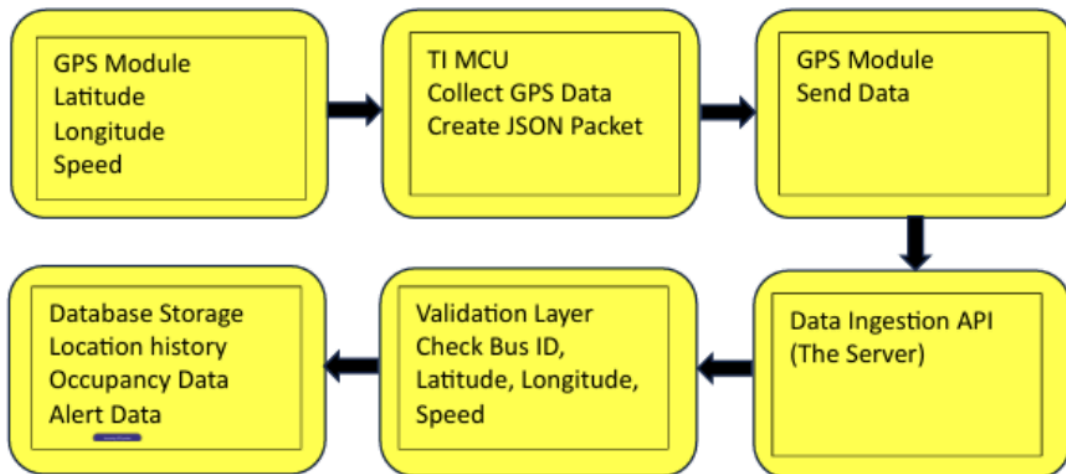


Diagram 2: Data Validation Flow

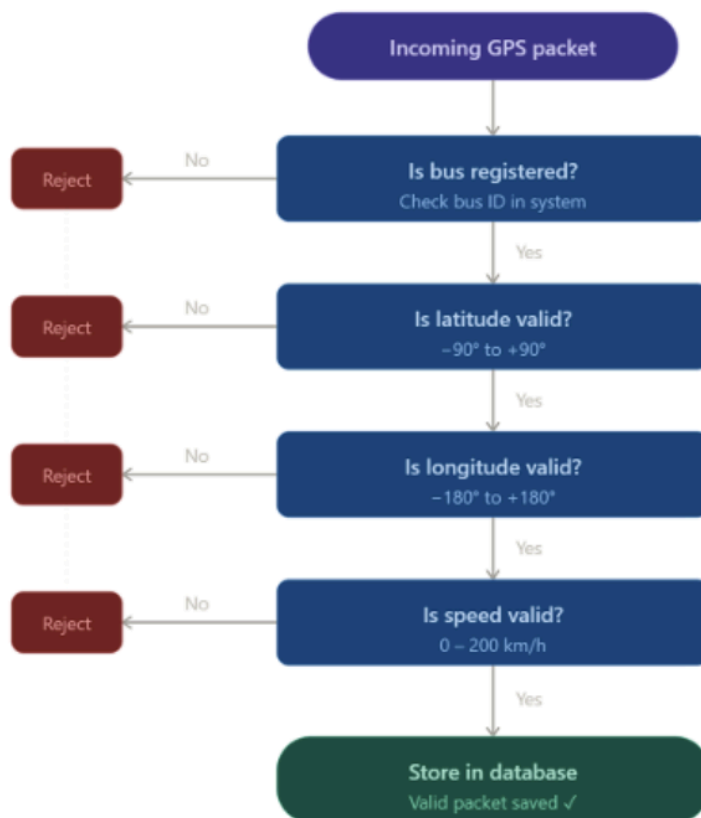
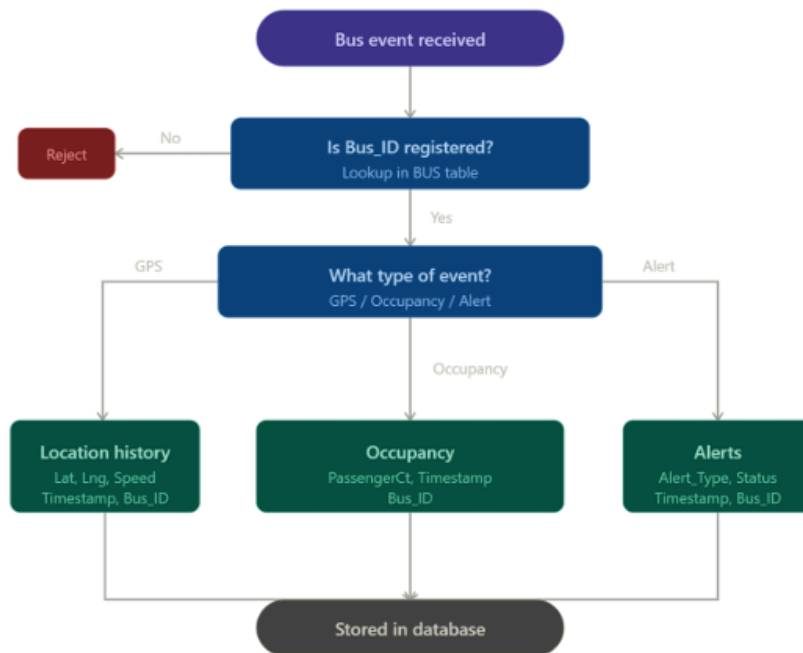
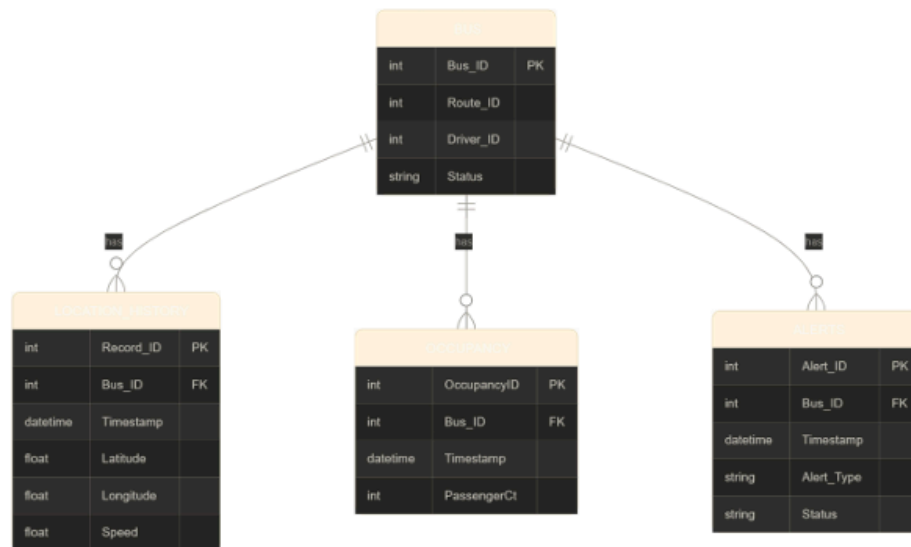
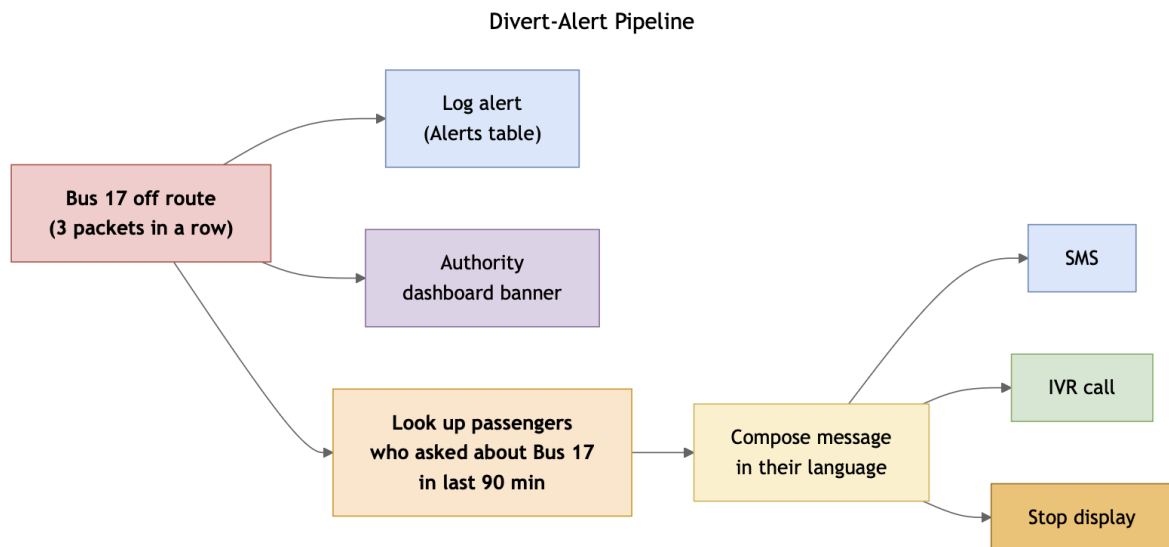
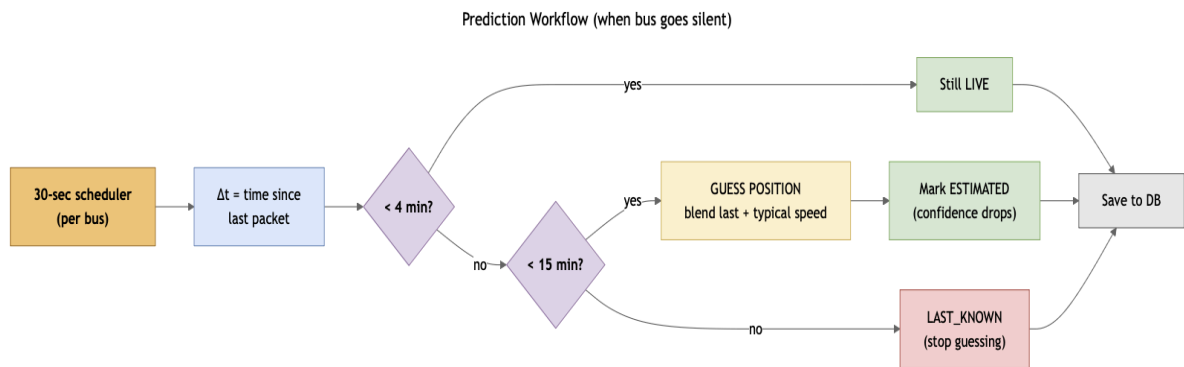
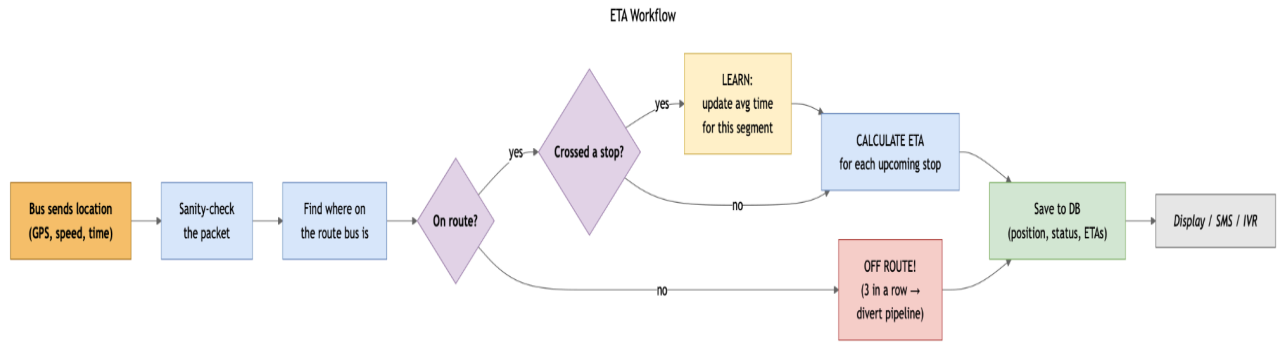


Diagram 3: Database Design (ER Diagram)





Geofencing – how the server decides if the bus is on the right route

THE IDEA

We draw an invisible 'corridor' around the planned route (say 50 m wide on either side of the road).

If the bus stays inside the corridor → on route.
If it goes outside for several updates in a row → diverted.

THE MATH (used to check this)

Distance between two GPS points
(Haversine formula)

Tells us 'how far is the bus from this point on the route?'
Accurate to a few metres – well under GPS error.

Perpendicular distance from bus to a road segment (Cross-track distance)

Tells us 'how far off the road line is the bus?'
This is the number we compare to the corridor width.

Fast version (good enough at our scale)

Convert lat/lon to flat x/y near the route, then standard high-school geometry to find the shortest distance from bus to road line.

~10× cheaper to compute, sub-metre error.

THE RULES (what the server actually does)

ON-ROUTE

Bus is within the corridor
(≤ 50 m from the road line)

ARRIVED at a stop

Bus is within 30 m of the stop AND barely moving

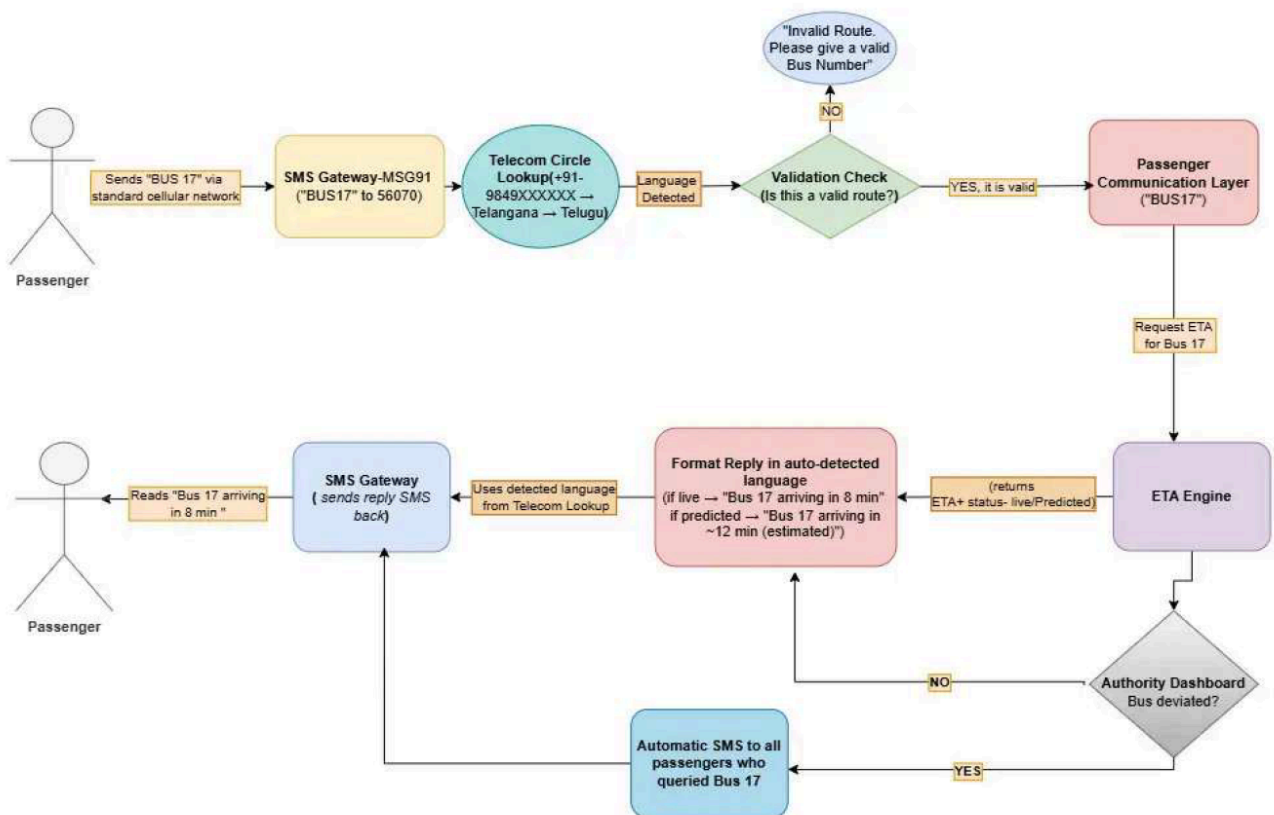
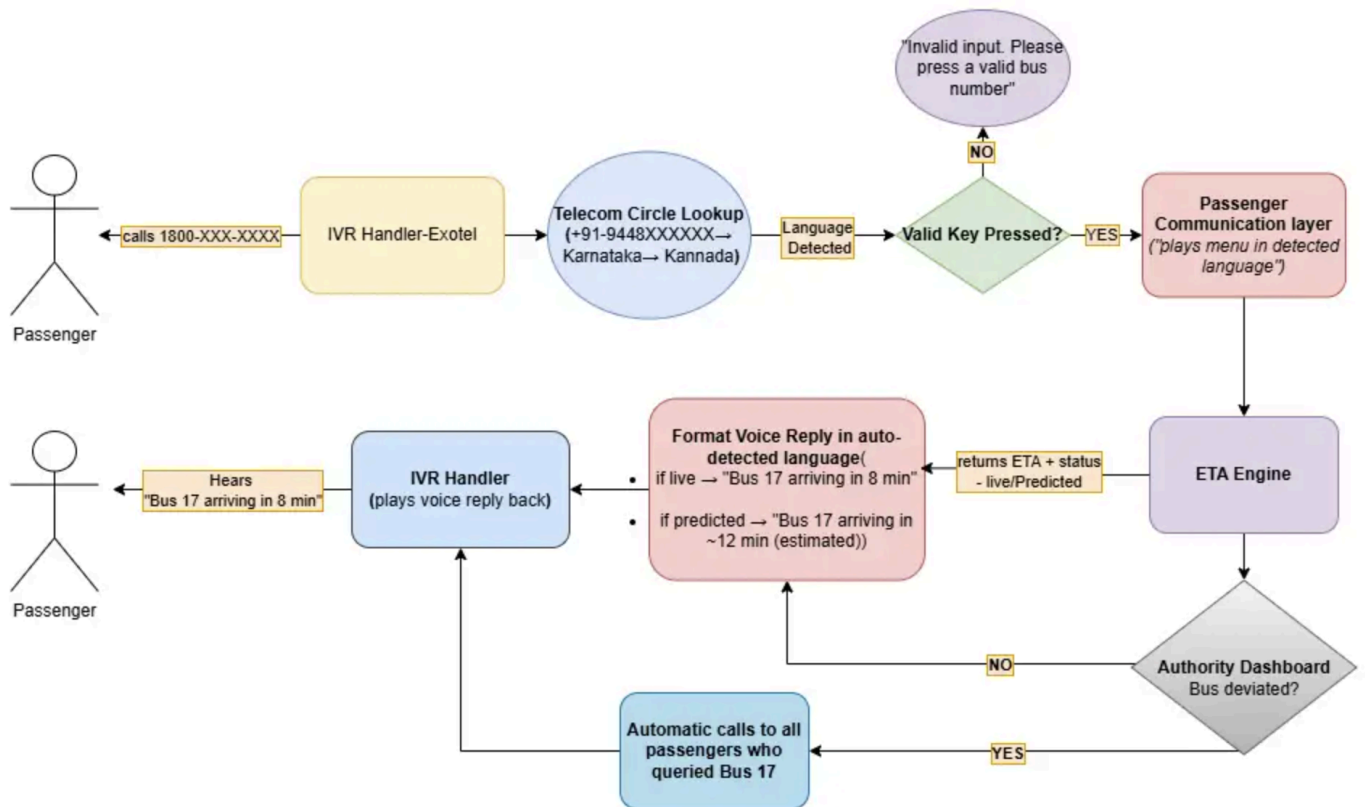
LEFT the stop

Bus is more than 50 m from the stop AND moving (extra 20 m so it doesn't flicker)

DIVERTED (off route)
Bus is outside the corridor for 3 packets in a row
(one bad GPS reading isn't enough – we wait for 3)

When diverted:

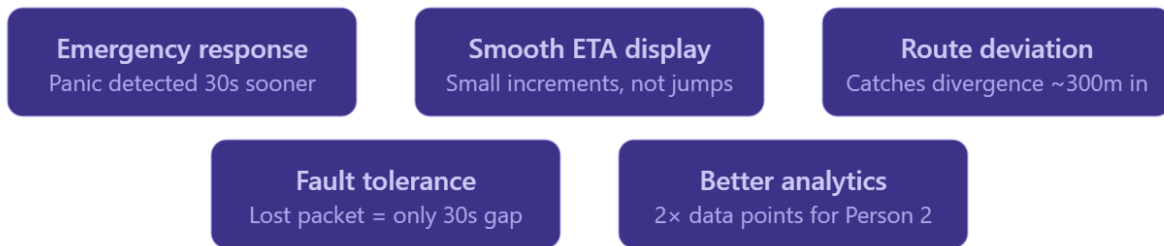
- log an alert in the database
- ping Person 3's dispatcher to notify passengers
- flash a banner on the authority dashboard



Why 30 seconds? Interval comparison

Interval	Location error (40 km/h)	Good for	Problem
1 sec	~11 m	Robotics, precision GPS	Huge data volume, GSM overload, expensive
10 sec	~111 m	High accuracy needed	Higher storage, battery, server load
30 sec ✓ chosen	~333 m	Public transport, ETA, smooth display updates	Balanced — accuracy vs cost ✓
1 min	~667 m	Low-cost IoT sensors	Slow emergency response, ETA jumps, stale data

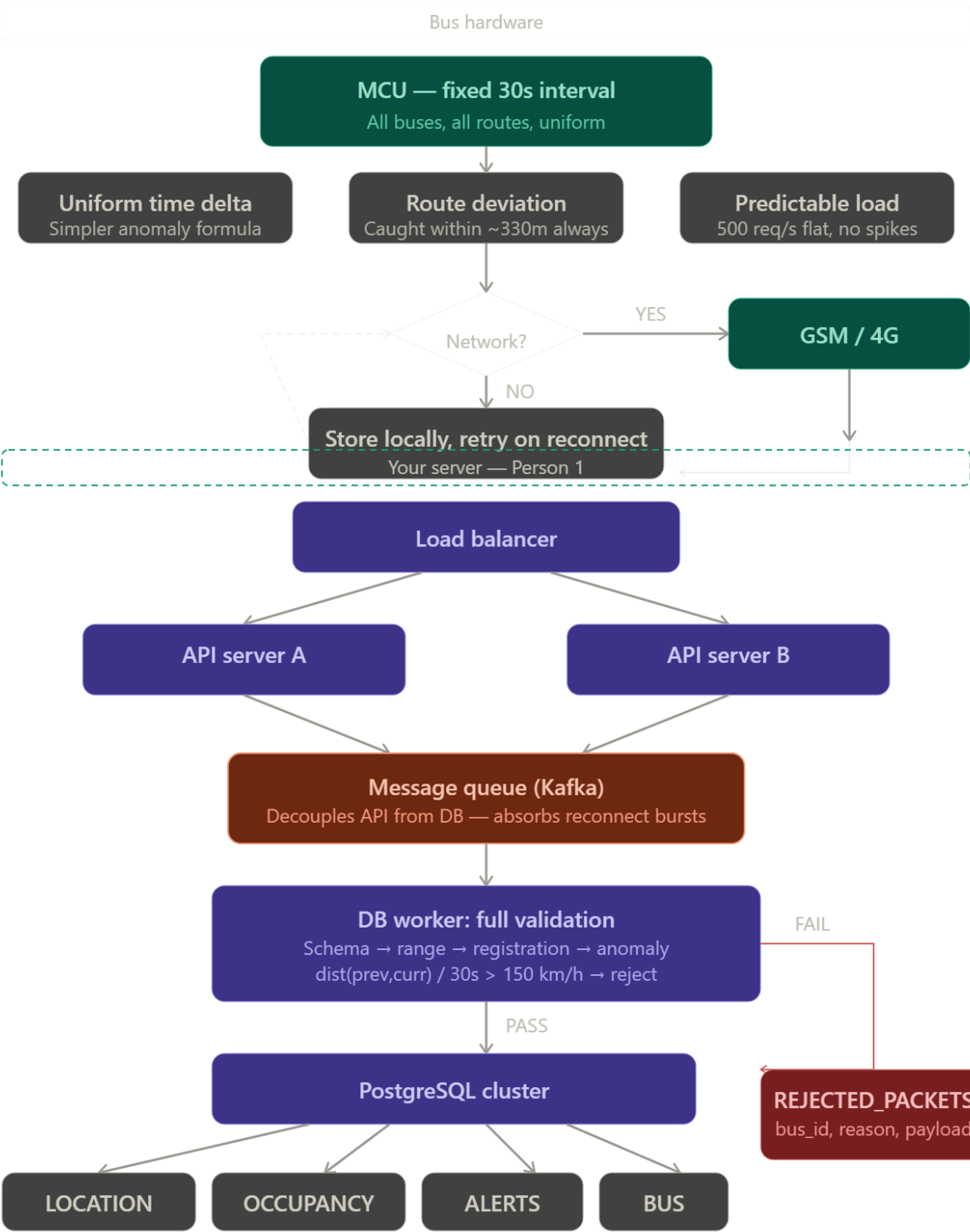
Five reasons 30s beats 1 min



Engineering rule: choose the coarsest interval that still meets all reliability requirements

Village-level X Fragmented	State-level ✓ Correct scope	National X Over-engineered
Data ownership RTC master data not local — needs remote JOINS per packet	BUS, ROUTE tables co-located with tracking DB — fast JOINS	20+ RTCs, different schemas, no agreed unified format
Query patterns Route R12 spans 14 villages → distributed JOIN across 14 DBs	Single indexed query: WHERE route_id='R12' Fast, consistent	Works, but requires per-RTC data isolation and translation layers
Consistency (CAP theorem) Bus crossing village boundary — ownership handoff window = stale ETA	One server always owns every bus. No handoff needed	Solvable with sharding by state, but that's state-level anyway
Governance (Indian Constitution) No legal authority at village level for inter-village routes	Transport = state subject (Schedule 7). KSRTC has full legal authority	Requires all RTCs to agree on data sharing — governance blocker
Scale 500+ village servers, each needs uptime, security, maintenance	15k buses, 500 req/s — modern cloud handles this easily	200k+ buses nationally 576M updates/day — unnecessary for KSRTC
Future scalability Cannot scale up — fragmentation gets worse, not better	Same codebase deployed per state — Karnataka then Telangana etc.	Correct end-state, but achieved via per-state instances, not one DB
Viva answer "Transport is a state subject. KSRTC's master data, route schema, and operational authority are all state-scoped — the tracking DB must be co-located with that data to avoid distributed JOINS." National scale = same platform, separate state instances — not one giant DB. Sharding by state IS state-level deployment with a common codebase.		

15,000 buses × 2 updates/min = 500 req/s sustained
43.2 million location updates per day



Hardware cost and size

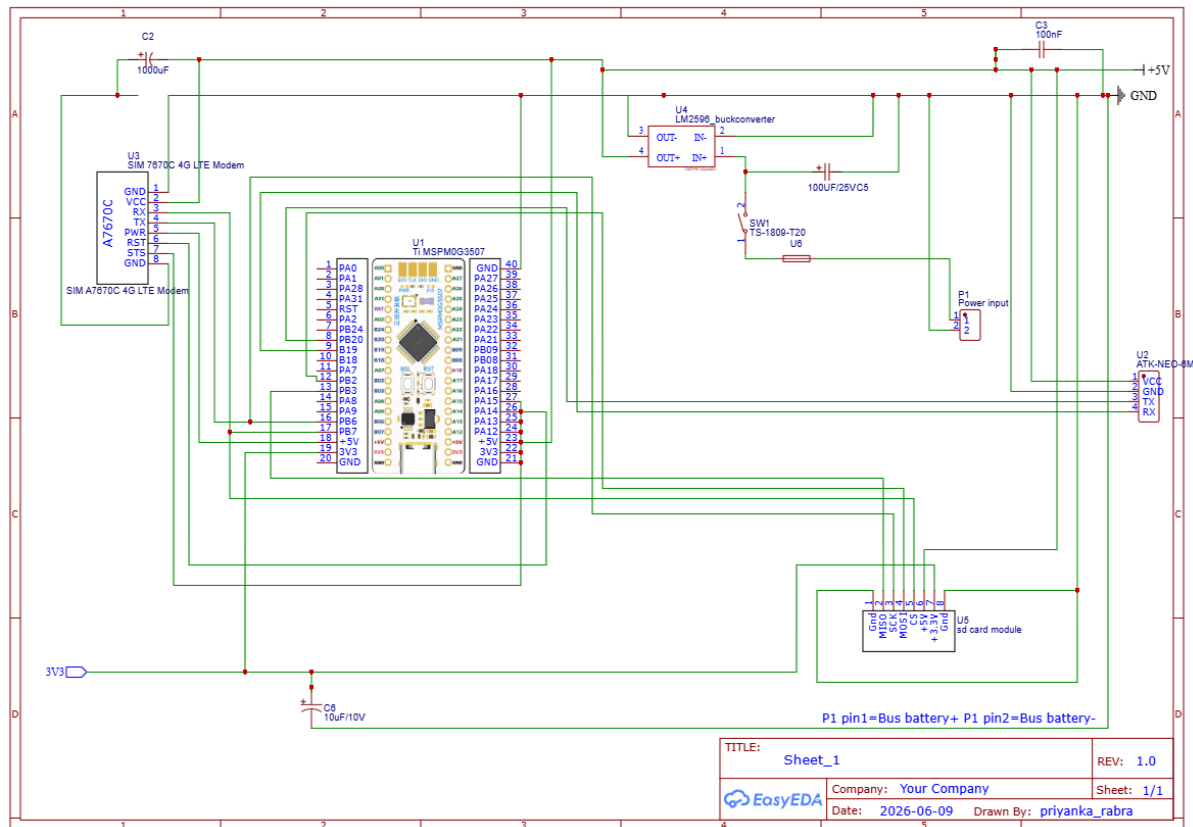
Cost and size :

Component	Approx Cost (India)	Size (Approx)	Important Parameters
NEO-6M GPS	₹300 to ₹600	25 × 25 mm	Accuracy, satellites, update rate, UART
MSPM0G3507 LaunchPad	₹2000–₹2500	65 × 50 mm	Flash, UARTs, ADC, GPIOs
A7670 LTE Module	₹1000–₹1500	30 × 30 mm (module)	LTE support, current consumption, SIM support
LTE Antenna	₹100–₹300	Varies Small whip antenna around 50-110mm long or PCB antenna around 40mm x 10mm	Signal strength
LM2596 Buck Converter	₹50–₹150	43 × 21 mm	Input range, output voltage, current capability
microSD card	600-800, for cost reduction we can use MCU flash		Memory storage, read speed
LM2596 buck converter(Ti)	50 rupees	43mm x 21mm x 14mm	
Total	4k to 6k		

Apart from the above additional capacitors and resistor, headers, breadboard.

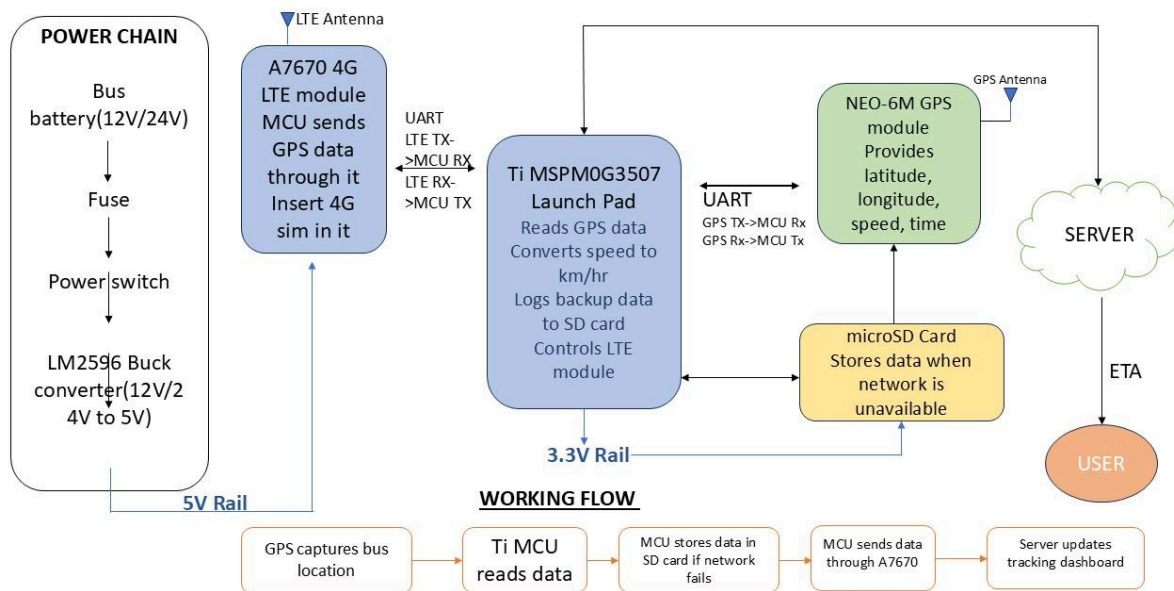
Considering all the sizes roughly approx total prototype size will be around 20 cm × 15 cm × 6 cm, including the MSPM0G3507 LaunchPad, A7670 LTE module, NEO-6M GPS module, buck converter, SD card module, jumper wires, and antennas. When using PCB approx enclosure size would be around 15cm x 10cm 5cm

Hardware working flow



This is a rough schematic for understanding basic connections. We might require more resistors, capacitors etc. Planning to add in the presentation after changes.

COMPONENT CONNECTION AND WORKING FLOW BLOCK DIAGRAM



Complete workflow(can be added in presentation)